
Documentación del Programa Clasificador de Gramáticas de Chomsky

Este documento detalla la estrategia de la solución implementada en C para clasificar una gramática formal según la Jerarquía de Chomsky (Tipos 0, 1, 2 y 3).

1. Estrategia de la Solución General

La estrategia de clasificación se basa en la **jerarquía estricta** de los cuatro tipos de gramáticas formales definida por Noam Chomsky, donde un tipo de gramática es un subconjunto estricto del tipo inferior (p. ej., toda gramática Tipo 3 es también Tipo 2, Tipo 1 y Tipo 0).

Por lo tanto, la clasificación se realiza mediante un proceso de **verificación descendente** (o de lo más restrictivo a lo menos restrictivo):

1. **Intentar clasificar como Tipo 3 (Regular):** Si todas las reglas cumplen las condiciones de una gramática regular, se clasifica como **Tipo 3** y el proceso termina.
2. **Intentar clasificar como Tipo 2 (Libre de Contexto):** Si falló el Tipo 3, se verifica la condición del Tipo 2. Si se cumple, se clasifica como **Tipo 2** y el proceso termina.
3. **Intentar clasificar como Tipo 1 (Sensible al Contexto):** Si falló el Tipo 2, se verifica la condición del Tipo 1. Si se cumple, se clasifica como **Tipo 1** y el proceso termina.
4. **Clasificar como Tipo 0 (Estructura de Frase):** Si la gramática no cumple ninguna de las restricciones anteriores, se clasifica por defecto como **Tipo 0**.

Para el caso del **Tipo 3**, se realiza una verificación adicional para determinar si es **Regular por Derecha** o **Regular por Izquierda**.

2. Representación de las Reglas de la Gramática

Las reglas de producción se representan en C mediante un *registro* (estructura) y se almacenan en un *arreglo unidimensional*.

Estructura Regla

Cada regla se almacena como una estructura que contiene dos campos, ambas cadenas de caracteres (`char[]`):

Campo	Propósito	Convención de Símbolos
izquierda (LHS)	Parte izquierda de la regla de producción.	Secuencia de No-Terminales (Mayúsculas) y Terminales (Minúsculas). Debe contener al menos un No-Terminal, es decir no vacía y no secuencia de terminales solos.
derecha (RHS)	Parte derecha de la regla de producción.	Secuencia de No-Terminales (Mayúsculas) y Terminales (Minúsculas), o la secuencia vacía, representada por "L" (Lambda).

Restricciones clave de entrada:

- **No-Terminales:** Letras mayúsculas (A-Z), excepto la **'L'** que se reserva para Lambda. El axioma es **'S'**.
 - **Terminales:** Letras minúsculas (a-z).
 - **Secuencia Vacía (lambda):** Representada por la cadena **"L"**. Solo puede aparecer sola en la parte derecha de una regla, que se conoce como regla de borrado.
-

3. Explicación del Código por Función

El programa utiliza varias funciones auxiliares para determinar las propiedades de los símbolos y las reglas, seguidas por las funciones de clasificación de cada tipo.

Funciones Auxiliares de Verificación

- `es_no_terminal(char c)`: Devuelve 1 si el carácter `c` es una letra mayúscula y no es 'L', indicando que es un **No-Terminal**.
- `es_terminal(char c)`: Devuelve 1 si el carácter `c` es una letra minúscula, indicando que es un **Terminal**.
- `es_secuencia_terminales(const char *s)`: Devuelve 1 si la cadena `s` no es "L" y consiste enteramente de terminales (minúsculas).
- `s_aparece_en_rhs(Regla g[], int n)`: Recorre todas las reglas y devuelve 1 si el axioma 'S' aparece como subcadena en la parte derecha de alguna regla. Esta función es crucial para verificar la excepción de borrado ($S \rightarrow L$ pero no aparece S a la derecha de ninguna regla) en el Tipo 1.

`clasificar_tipo_3(Regla g[], int n)`

Verifica las restricciones más estrictas del Tipo 3 (Regular). El tipo 3 exige:

- **LHS**: Debe ser un **único No-Terminal** (A).
- **RHS**: Debe ser una de las formas:
 - Una secuencia de terminales (`w`).
 - Una secuencia de terminales seguida de un solo No-Terminal (`wN`).
 - Un solo No-Terminal seguido de una secuencia de terminales (`Nw`).
 - Como caso particular se admiten reglas de borrado y red denominación, es decir cuando `w` es vacía.

Esta función utiliza dos banderas (`es_regular_derecha`, `es_regular_izquierda`) para asegurarse de que la gramática sea **uniformemente** derecha o izquierda, sin mezclarlas (excepto para las reglas $A \rightarrow t$ o $A \rightarrow L$ que son válidas en ambos). Retorna 'D', 'I' o 'N' (No Regular).

`es_tipo_2(Regla g[], int n)`

Verifica las restricciones del Tipo 2 (Libre de Contexto). El tipo 2 exige:

- **LHS**: Debe ser un **único No-Terminal** (A).
- **RHS**: Cualquier secuencia de símbolos terminales y/o no-terminales, incluyendo "L" (regla de borrado) y un no-terminal solo (regla de red denominación).

La función simplemente itera sobre todas las reglas y comprueba que la longitud de LHS sea 1 y que el símbolo sea un No-Terminal.

`es_tipo_1(Regla g[], int n)`

Verifica las restricciones del Tipo 1 (Sensible al Contexto). El tipo 1 exige:

- **LHS y RHS**: Se debe cumplir que la longitud del lado izquierdo sea **menor o igual** a la longitud del lado derecho ($|\alpha| \leq |\beta|$).
- **Excepción de borrado**: La única regla que puede violar la condición de longitud es el borrado del axioma ($S \rightarrow L$) y esto solo es permitido si **S no aparece en el RHS** de ninguna regla.

La función mantiene la bandera `tiene_s_a_lambda` y, al final, llama a `s_aparece_en_rhs` para verificar la excepción.

`clasificar_gramatica(Regla g[], int n, char *tipo_3_regularidad)`

Esta es la función principal de clasificación que implementa la estrategia descendente:

```
// Verifica T3
if (clasificar_tipo_3() != 'N') return 3;

// Verifica T2
if (es_tipo_2()) return 2;
```

```
// Verifica T1
if (es_tipo_1()) return 1;

// Si todo falló
return 0; // Tipo 0
```

4. Ejemplo de Ejecución (Casos Posibles)

La siguiente tabla muestra ejemplos de entrada de reglas y la salida esperada por el programa para cubrir todos los casos de la jerarquía.

Tipo de Gramática	Entrada del Usuario	Salida del Programa
Tipo 3 (Regular por Derecha)	S -> aB	3
	B -> b	Clasificación Detallada: Regular por Derecha
	B -> L	
Tipo 3 (Regular por Izquierda)	S -> Baa	3
	S -> c	Clasificación Detallada: Regular por Izquierda
	B -> Bb	
Tipo 2 (Libre de Contexto)	S -> aSa	2
	S -> b	
	A -> AB	
Tipo 1 (Sensible al Contexto)	S -> aBC	1
	S -> L	
	CB -> BC	
	aB -> aba	
Tipo 0 (Irrestricta)	S -> ABC	0
	B -> b	
	AB -> a	
Tipo 0 (Falla Excepción $S \rightarrow L$)	S -> aBC	0
	S -> L	
	aB -> aSa	

Ejemplo de Interacción (Gramática Tipo 1)

Interacción en Consola:

--- Clasificador de Gramáticas de Chomsky ---

Convenciones: No-Terminales=Mayúsculas excepto L (S=Axioma), Terminales=Minúsculas.

La secuencia vacía Lambda se representa con L

Ingrese las reglas en formato: IZQUIERDA -> DERECHA

Coloque un espacio antes y después del operador ->

Escriba 'FIN' para terminar la entrada.

Regla 1: S -> aBC

Regla 2: S -> L

Regla 3: CB -> BC

Regla 4: aB -> aba

Regla 5: FIN

--- RESULTADO DE LA CLASIFICACIÓN ---

Tipo de Gramática (0-3): **1**
